

Milestone 3

Members: Nitul Patel and Nikhil Patel

```
import pandas as pd

df1 = pd.read_csv("pdata.csv")
df2 = pd.read_csv("sdata.csv")
```

The target variable in this project is commits, as it represents developer productivity and performance. We used three methods to reduce the dimensionality of the dataset in this project: correlation analysis, feature selection, and Principal Component Analysis. These methods helped us focus on the most important information.

```
# Method 1: Correlation
corr = df2.corr()
print(corr['commits'].sort_values(ascending=False))

commits      1.000000
hours_coding  0.644200
coffee_intake_mg  0.554904
ai_usage_hours  0.378334
task_success  0.336383
cognitive_load  0.083160
bugs_reported  0.026582
distractions  -0.042403
sleep hours   -0.046093
Name: commits, dtype: float64

# Method 2: Select Top Columns
important_cols = ['ai_usage_hours', 'hours_coding', 'commits']
df2_selected = df2[important_cols]
print(df2_selected.head())

   ai_usage_hours  hours_coding  commits
0             0.71           5.99      2.0
1             1.75           4.72      5.0
2             2.27           6.30      2.0
3             1.40           8.05      9.0
4             1.26           NaN       4.0

df1 = df1.dropna()
df2 = df2.dropna()
```

```

# Method 3: PCA
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

# select numeric data
num_data = df2.select_dtypes(include='number')

# scale data
scaled = StandardScaler().fit_transform(num_data)

# apply PCA
pca = PCA(n_components=2)
pca_data = pca.fit_transform(scaled)

print(pca.explained_variance_ratio_)

[0.35787221 0.23597649]

```

Justification and discussion on dimension reduction techniques

We used three different ways to reduce the number of dimensions in the dataset: correlation analysis, feature selection, and Principal Component Analysis (PCA). This helped us focus on the most important information.

We used correlation analysis to figure out how different variables are related to each other. It helped us figure out which features are very useful and which ones aren't. This helped us get rid of extra columns and keep only the ones that were important.

Feature selection was used to pick the most important columns by hand, like AI usage hours, coding hours, and commits. We chose these features because they directly show how active and productive developers are. This step helps make the data smaller and the analysis faster.

We used Principal Component Analysis to cut down on the number of variables by turning the dataset into a smaller set of components that still held most of the important information. This makes it easier to work with complicated data and speeds things up.

We chose these methods because they are easy to use, work well, and fit our dataset. They make things less complicated, speed up analysis, and make sure that only useful features are used for further analysis.

Important features

The most important columns in the secondary dataset are AI usage hours, commits, and coding hours. These columns show what developers are doing and how well they are doing it. AI usage hours show how often developers use AI tools, coding hours show how long they spend on development work, and commits show how much work they have done. These features, when combined, give a clear picture of how productive and efficient someone is.

The most important parts of the primary dataset are tweets and labels because they show how people feel about ChatGPT. The tweets have the actual text data that shows what users are

thinking, and the labels show how they feel (positive, negative, or neutral). These features are necessary for sentiment analysis and for figuring out how people feel about AI tools.

```
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score

# Converted text to numbers
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(df1['tweets'])

y = df1['labels']

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2)

# Train model
model = MultinomialNB()
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))

Accuracy: 0.7152271068952937
```

Justification and discussion on sentiment analysis model

For sentiment analysis on the main dataset in this project, we used the Naive Bayes (MultinomialNB) model. We chose this model because it is simple, quick, and very good at classifying text, like figuring out how someone feels. It works well with a lot of text data and is often used to sort text into groups like positive, negative, and neutral.

Count Vectorizer changed the text data (tweets) into numbers before the model was used. This step changes words into numbers based on how often they are used, which helps the model understand and process the text. After that, the dataset was split into training and testing sets so that the model could be tested.

The Naive Bayes model is simple but still works well for many real-world uses because it assumes that words are not related to each other. It worked well for us and gave a good accuracy score, which shows that it can correctly figure out how users feel.

Overall, this model was a good fit for our project because it is easy to use, doesn't need as much processing power, and works well for sentiment classification tasks.

Team tasks

Nitul Patel (member 1), was in charge of preparing the data and the model. This involved filling in missing values, cleaning up the datasets, using dimensionality reduction methods like correlation analysis, feature selection, and PCA, and getting the data ready for more analysis.

Nikhil Patel (member 2), was in charge of putting the sentiment analysis model into action and testing it. This involved using the Naive Bayes model, turning text data into numbers with Count Vectorizer, making predictions, and figuring out how accurate the model was. The member also worked on figuring out what the results meant and making visual outputs.

